Final Report: Roman Numeral Prediction Challenge

1. Project Title

Roman Numeral Prediction Using CNNs with PyTorch and Flask

2. Team Members

- Dara Shikoh Bodla [2023176]
- Marwah Mohammad [2023304]

3. Objective

To build a machine learning system that accurately identifies handwritten Roman numerals (I to X) from uploaded images. The model is trained using a supervised learning approach and deployed with a web interface for real-time predictions.

4. Roman Numeral Classes

["I", "II", "III", "IV", "IX", "V", "VI", "VII", "VIII", "X"]

5. Methodology

5.1 Supervised Learning Approach

Used labeled training data where each image corresponds to one of 10 Roman numeral classes. A Convolutional Neural Network (CNN) was trained using these labels to minimize classification error. The model learns patterns and features from image pixels and uses these to predict class probabilities.

5.2 Dataset and Preprocessing

Dataset directory structure follows ImageFolder format from PyTorch. Images were resized to 64×64, converted to tensors, and normalized to mean=0.5, std=0.5.

5.3 Model Architecture

A custom CNN model RomanCNN was built with:
- Convolutional Layers: Conv2D → ReLU → MaxPooling (×3 blocks with increasing depth: 32, 64, 128)
- Fully Connected Layers: Flatten → Linear(8192→256) → Dropout(0.3) → Linear(256→10)
- Activation: ReLU in all layers; Softmax applied at inference.

6. Training Details

Parameter	Value
Optimizer	Adam
Loss Function	CrossEntropyLoss
Epochs	10
Batch Size	32
Train/Val Split	80/20
Accuracy (Final Epoch)	~92–95% (train set)
Model File	roman_model.pth

7. Flask Web Application

Features:
- Built using Flask
- Routes: '/' (Upload page and result display), '/predict' (Returns JSON prediction via POST)
- Model is loaded in CPU mode and used for inference on uploaded images
- Predictions returned using:

```
_, predicted = torch.max(output, 1)
prediction = class_names[predicted.item()]
```

8. Results

High accuracy across all classes. Minor confusion among similar-looking numerals (e.g., “VII” vs “VIII”). Real-time web inference is efficient and responsive.

9. Challenges Faced

- Visual similarity among Roman numeral characters caused occasional misclassifications
- Ensuring the model generalizes well despite handwriting variations
- Flask integration with PyTorch and handling image uploads robustly

10. Conclusion

This project demonstrates a robust and accurate CNN-based Roman numeral recognition system using supervised learning. With a clean web interface, users can easily upload handwritten numeral images and receive instant predictions. This system can be extended to recognize more symbols or numerals with dataset expansion and augmentation.

11. Tools & Technologies

- Language: Python
- Libraries: PyTorch, TorchVision, Flask, PIL
- IDE: VS Code / Jupyter
- Hardware: CPU/GPU (if available)

12. Future Enhancements

- Add camera input or drag-and-drop feature to web UI
- Apply data augmentation to improve model robustness
- Provide confidence scores alongside predictions
- Deploy the app using services like Heroku or Render